

WEEK 02

그래픽스 수학 기초

벡터 · 행렬 · 좌표계 변환의 이해와 Unity Transform 실습

컴퓨터그래픽스 VF3500076 · 디자인엔테크놀로지전공

조성호 · 2026학년도 2학기

도입 질문

인스펙터에 **Rotation (90, 90, 0)** 입력
옆 사람도 같은 값 입력

결과: **두 물체가 서로 다른 방향**
이유는?

도입

학습 목표

1 원리

기저 벡터의 이동 위치로 행렬 직접 유도 — 암기 불필요

2 함정

AI 회전 코드의 **짐벌락** 증상 확인 →
원인 지목

3 VR

VR의 **통제 불가 변환** : 사용자의 머리
→ 직접 조작 시 멀미 유발

PART I · 60분

벡터, 행렬, 그리고 변환

화면 속 모든 움직임 = 4×4 행렬 곱셈

점과 벡터의 차이

점 (Point)

위치 — 어디에 있는가

원점 이동 시 값 변경

벡터 (Vector)

방향과 크기 — 어느 쪽으로 얼마나

원점 이동과 무관

Unity에서는 둘 다 `Vector3` 타입 → 구분이 흐려짐
이동 변환 적용 시 동작 차이 발생 (뒷부분에서 다시 다룸)

Unity의 좌표계

Y ↑ 위 (up)
 Z → 앞 (forward)
 X → 오른쪽 (right)

왼손 좌표계 (left-handed)

왼손의 엄지=X, 검지=Y, 중지=Z 방향 = Unity의 축

도구	위	좌표계
Unity	Y	왼손
Unreal	Z	왼손
Blender	Z	오른손
Maya	Y	오른손
OpenGL	Y	오른손

Blender 모델을 Unity로 가져올 때 축이 늡는 원인

내적 — 두 방향이 같은 쪽을 보는 정도

내적 — 두 방향이 같은 쪽을 보는 정도



$$a \cdot b = |a| |b| \cos \theta$$

$$= |a| \times (\text{투영 길이})$$

$a \cdot b > 0$	앞쪽
$a \cdot b = 0$	정확히 직각
$a \cdot b < 0$	뒤쪽

내적·외적 직접 조작

인터랙티브 데모

이 자리에는 직접 조작하는 3D 데모

HTML 슬라이드를 브라우저로 열어 확인

청록 = 내 시선 a · 분홍 = 대상 방향 b · 노랑 = 외적 $a \times b$

내적의 활용

```
// 적이 내 시야각(60°) 안에 있는가?  
Vector3 toEnemy = (enemy.position - transform.position).normalized;  
float d = Vector3.Dot(transform.forward, toEnemy);  
  
if (d > Mathf.Cos(60f * 0.5f * Mathf.Deg2Rad)) // = 0.866  
    Debug.Log("보인다");
```

각도 계산 없이 비교 — **Acos** 는 느리고 불필요

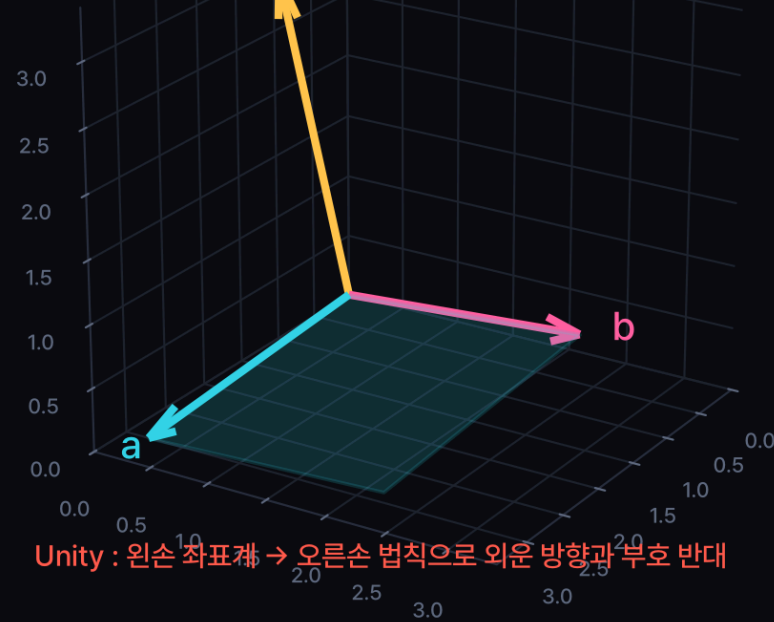
코사인 값끼리 비교로 충분 → 그래픽스에서 내적을 활용하는 이유

외적 — 두 벡터가 만드는 면

외적 — 두 벡터가 만드는 면

$|a \times b|$ = $a \times b$ 평행사변형의 넓이

$a \times b$ 는 a, b 양쪽에 수직 $\rightarrow$ 면의 법선(normal)



외적의 활용

면의 법선

삼각형 세 점 → 면의 방향 계산 → 조명 계산의 출발점(5주차)

회전축

"a를 b로 돌리려면 어느 축으로?" → $\mathbf{a} \times \mathbf{b}$. 쿼터니언의 방식

좌우 판정

대상이 내 왼쪽인지 오른쪽인지 판정 (내적으로는 불가)

내적만으로는 좌우 구분 불가 — 왼쪽 30°와 오른쪽 30°의 내적 값 동일
방향까지 필요할 때 외적 활용

변환의 세 가지 종류

변환	역할	원점	Unity
Scale	크기	고정	localScale
Rotate	방향	고정	rotation
Translate	위치	이동	position

Transform 컴포넌트의 세 줄이 전부 → 나머지는 이 셋의 조합

마지막 줄의 «원점 이동» → 이후 문제의 원인

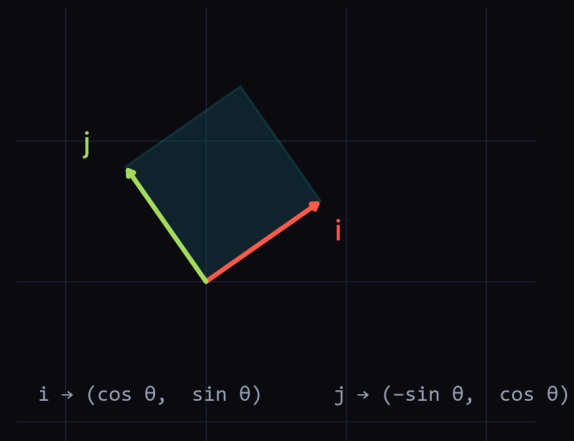
행렬의 구성 원리

행렬의 열 = 기저 벡터가 도착한 자리

변환 전



35° 회전 후



$$R = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}$$

← 1열 = i가 도착한 자리
← 2열 = j가 도착한 자리

행렬 암기 불필요 — 기저 벡터의 도착 위치만 알면 행렬 유도 가능

회전행렬 유도

$i = (1,0)$ 을 θ 만큼 회전하면?

$$i \rightarrow (\cos \theta, \sin \theta)$$

$j = (0,1)$ 을 θ 만큼 회전하면?

$$j \rightarrow (-\sin \theta, \cos \theta)$$

두 결과를 **열로 세운 것** = 행렬

$$R = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}$$

공식 암기 없이 **언제든 다시 유도 가능**

문제 — 선형변환으로 표현할 수 없는 이동

문제 — 2D 행렬로 이동 표현 불가

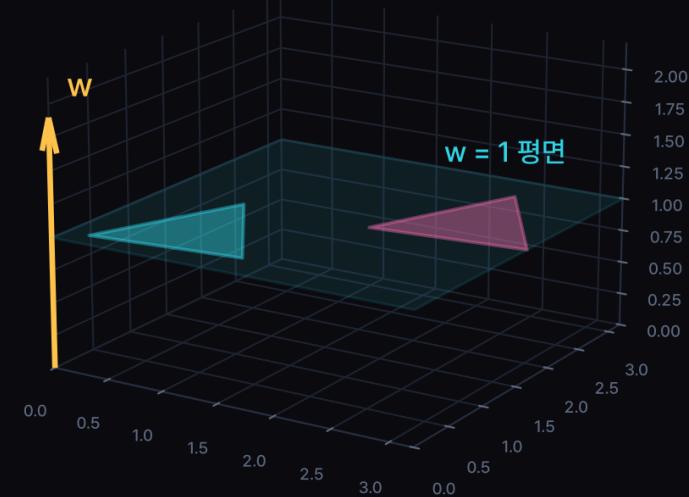
회전·스케일 : 원점이 제자리

이동 : 원점 이동 → 선형변환 불가



해결 — 한 차원 추가 $(x, y) \rightarrow (x, y, 1)$

3D 의 전단(shear) = 2D 의 이동



점은 $(x, y, z, 1)$ · 방향은 (x, y, z, θ) — 방향 벡터는 이동의 영향 없음

한 차원 추가로 이동 함수

$$\left[\begin{array}{cccc|c} 1 & 0 & 0 & tx & \\ 0 & 1 & 0 & ty & \\ 0 & 0 & 1 & tz & \\ 0 & 0 & 0 & 1 & \end{array} \right]$$

이동 · 회전 · 스케일 · 투영이
모두 같은 **4×4 곱셈으로 통일**

점은 **w=1** · 방향은 **w=0**

▶ 애니메이션 재생은 HTML 슬라이드에서

동차좌표 — 4×4 행렬을 활용하는 이유

$$\begin{array}{c}
 \left[\begin{array}{cccc} 1 & 0 & 0 & tx \\ 0 & 1 & 0 & ty \\ 0 & 0 & 1 & tz \\ 0 & 0 & 0 & 1 \end{array} \right] \begin{array}{c} x \\ y \\ z \\ 1 \end{array} = \begin{array}{c} x+tx \\ y+ty \\ z+tz \\ 1 \end{array}
 \end{array}$$

마지막 성분 $w = 1 \rightarrow$ 4번째 열(tx, ty, tz) 활성화
 이동·회전·스케일·투영을 모두 같은 4×4 곱셈으로 통일

$w = 1$ 은 점, $w = 0$ 은 방향

점 (x, y, z, 1)

이동 적용
물체의 위치, 정점 좌표

방향 (x, y, z, 0)

이동 무시
법선, forward, 빛의 방향

```
Matrix4x4 m = transform.localToWorldMatrix;
```

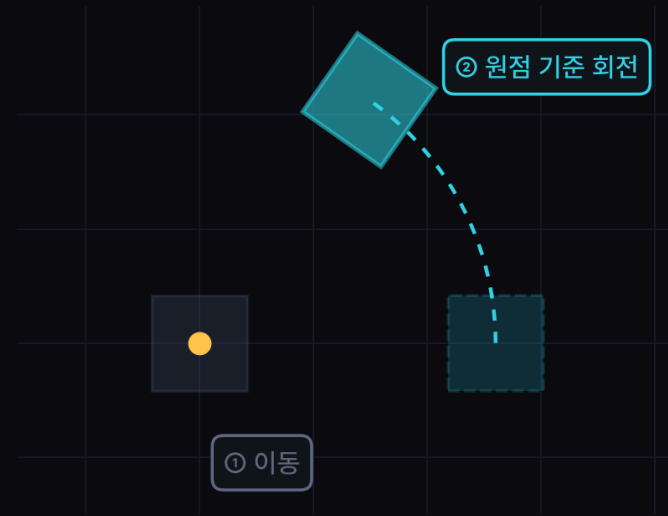
```
m.MultiplyPoint(v);    // w=1 로 취급 - 이동 포함  
m.MultiplyVector(v);   // w=0 로 취급 - 이동 무시
```

법선에 **MultiplyPoint** 적용 시 원점에서 멀어질수록 조명 이상

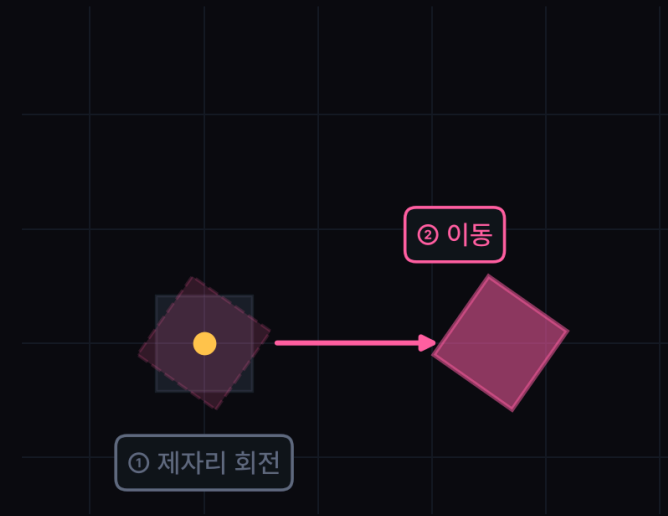
5주차 실습에서 재현

같은 R, 같은 T, 다른 순서

$R \cdot T$ (이동 → 회전)



$T \cdot R$ (회전 → 이동)



같은 R, 같은 T — 순서만 바뀌어도 결과 차이 → 행렬 곱의 교환법칙 불성립

Unity 의 `transform.Rotate()` 와 `transform.RotateAround()` 의 차이도 이 순서 문제

슬라이더 조작으로 확인

인터랙티브 데모

이 자리에는 직접 조작하는 3D 데모

HTML 슬라이드를 브라우저로 열어 확인

청록 = $R \cdot T$ (이동 먼저) · 분홍 = $T \cdot R$ (회전 먼저)

행렬 곱의 교환법칙 불성립

$$R \cdot T \neq T \cdot R$$

읽는 방향 주의 :

$M = R \cdot T \rightarrow$ 점에 T 를 먼저, 그 다음 R 을 적용
(오른쪽 행렬부터 적용)

Unity의 TRS 순서 : **Scale** → **Rotate** → **Translate**

행렬로는 $M = T \cdot R \cdot S \rightarrow$ 스케일이 가장 먼저 적용

코드에서의 차이

제자리 회전

```
transform.Rotate(  
    Vector3.up, 55f);
```

```
// 위치 그대로, 방향만  
// =  $T \cdot R$ 
```

원점 기준 회전

```
transform.RotateAround(  
    Vector3.zero,  
    Vector3.up, 55f);
```

```
// 위치까지 변경  
// =  $R \cdot T$ 
```

AI에게 "물체를 회전시켜줘" 만 요청 → 둘 중 어느 쪽을 고를지 알 수 없음

회전 중심 명시 = 프롬프트의 역할

AI 함정 #1

짐벌락

"부드럽게 회전시켜줘" 요청 시 AI가 반복하는 실수

오일러각 — 인스펙터의 세 숫자

세 축을 차례로 회전 · 직관적이고 읽기 쉬움

Rotation X: 30 Y: 45 Z: 0
 피치 요 롤

문제는 «차례» — 순서를 정하지 않으면 같은 숫자에서 다른 결과

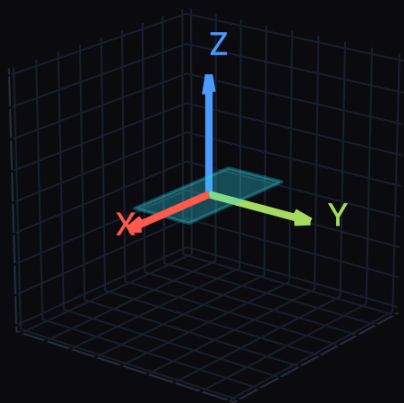
Unity의 적용 순서 : $Z \rightarrow X \rightarrow Y$

도입 질문 «같은 (90, 90, 0)인데 왜 다른가» 의 절반에 대한 답

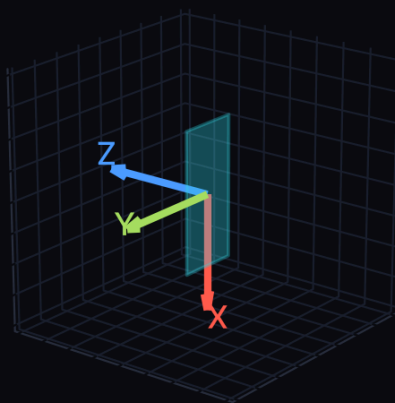
회전 순서에 따른 결과 차이

오일러각 — 순서 지정 없이는 의미 없음

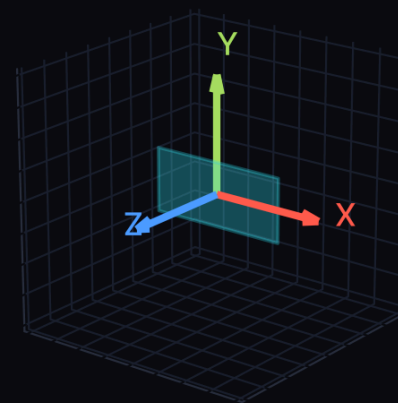
시작



X 90° → Y 90°



Y 90° → X 90°



같은 (90°, 90°) 에서 다른 결과 → '회전 순서(rotation order)' 지정 필요

Unity 인스펙터 Rotation 적용 순서 : Z → X → Y

짐벌락 — 회전축 하나의 상실

세 개의 링이 중첩된 장치 : 바깥부터 **Y** → **X** → **Z**

가운데 링(**X**, 피치)을 **90°** 로 세우는 순간

바깥 링(**Y**)과 안쪽 링(**Z**)이 같은 축으로 회전

회전 손잡이는 세 개, 실제 가능한 회전은 2차원 → **자유도 3 → 2**

오일러각 표현 방식 자체의 구조적 한계
어떤 순서를 선택해도 반드시 발생하는 지점 존재

피치 90° 조작

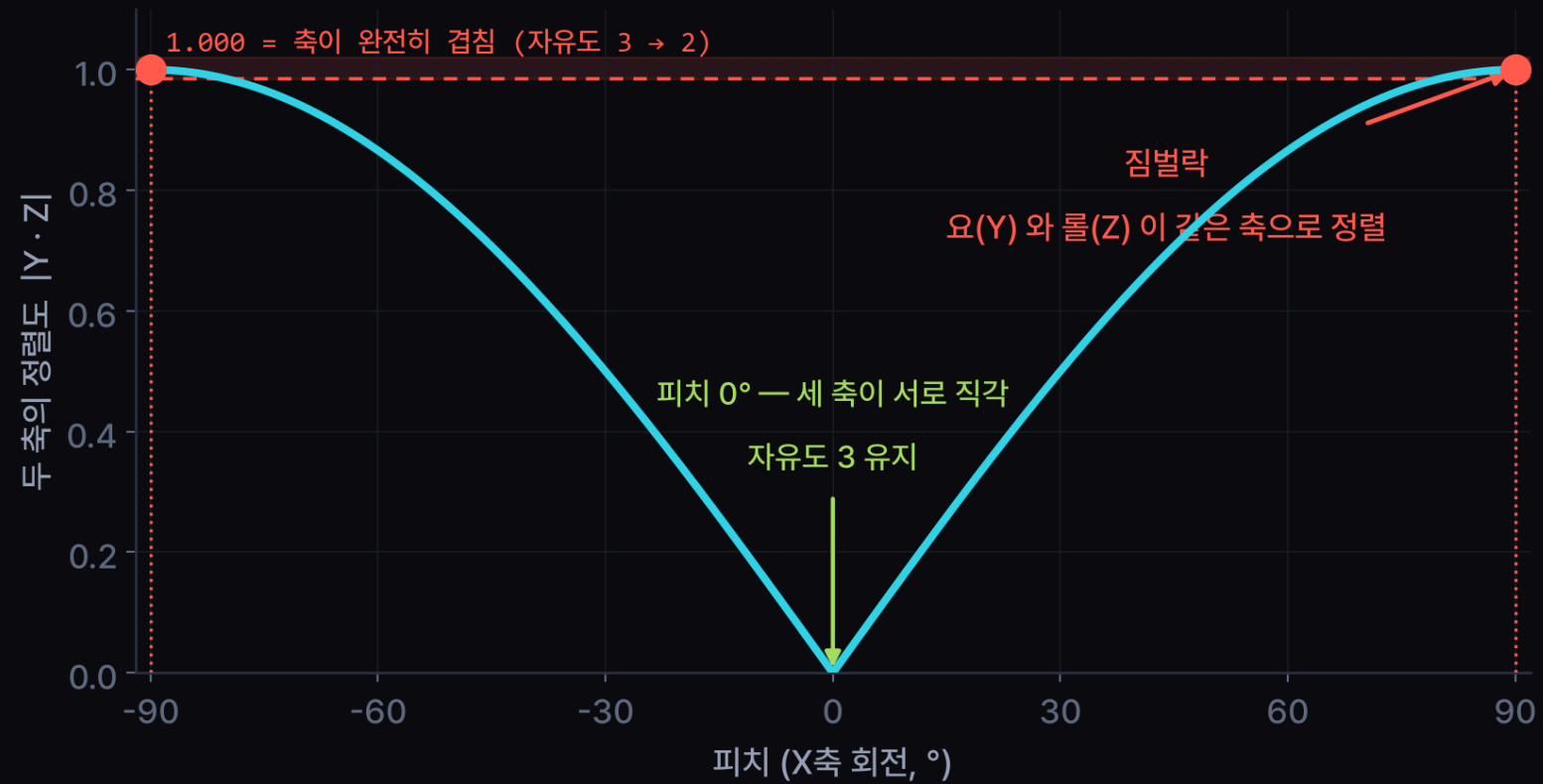
인터랙티브 데모

이 자리에는 직접 조작하는 3D 데모

HTML 슬라이드를 브라우저로 열어 확인

짐벌락 = 구조적 결함

짐벌락 — 오일러각의 구조적 결함



쿼터니언 — 네 개의 숫자로 표현하는 회전

$$q = (x, y, z, w)$$

축 (x, y, z) 둘레로

각도 θ 만큼 한 번에 회전

«세 축을 차례로» 대신 «임의의 축 하나를 중심으로 한 번에»

차례 없음 → 순서 문제와 짐벌락 없음

	오일러	쿼터니언
짐벌락	있음	없음
보간	덜컹거림	매끄러움
누적 오차	쌓임	정규화로 해결
사람이 읽기	쉬움	불가능

Unity : 내부는 쿼터니언, 인스펙터 표시만 오일러각

Unity 실무 규칙 네 줄

```
// 만들 때 - 사람이 읽는 각도로  
transform.rotation = Quaternion.Euler(0, 45, 0);  
  
// 거냥할 때 - 방향 벡터에서 바로  
transform.rotation = Quaternion.LookRotation(target - transform.position);  
  
// 부드럽게 - 반드시 Slerp  
transform.rotation = Quaternion.Slerp(a, b, t);  
  
// 절대 하지 말 것  
transform.eulerAngles += new Vector3(0, speed * Time.deltaTime, 0);
```

마지막 줄 = **AI가 가장 자주 작성하는 코드**

`eulerAngles` 는 읽을 때마다 쿼터니언에서 다시 계산 → 누적 시 값이 튀고 짐벌락 발생

부모-자식 = 행렬 곱의 연쇄

$$M_{world} = M_{부모} \cdot M_{자식} \cdot M_{손자} \cdot \dots$$

localPosition

부모 기준 · 부모 이동 시 함께 이동

position

월드 기준 · 체인 전체를 곱한 결과

```
transform.TransformPoint(localPos);      // 로컬 → 월드  
transform.InverseTransformPoint(worldPos); // 월드 → 로컬
```

비균등 스케일 + 회전 = 찌그러짐

부모에 `Scale (1, 3, 1)` 적용 후 자식 회전 → 자식이 **전단(shear)** 되어 기울어짐

$$S \cdot R \neq R \cdot S$$

스케일과 회전도 순서 교환 불가

Unity 콘솔 경고 없음 → 결과만 이상해짐

해결

스케일이 필요하면 **빈 부모** 추가 → 부모에서 회전, 자식에서 스케일
또는 균등 스케일만 사용

VR의 손 모델·UI 스케일 조정 시 자주 발생

PART II · 30분

그리고 VR에서는

변환 체인에 끼어드는 통제 불가 요소 — 사용자의 욕

지각 · 1

VR에서 카메라를 움직이는 주체는?

일반 게임은 개발자. VR은?

VR의 좌표계 체인

VR의 좌표계 체인 — 조작 가능한 변환과 조작 금지 변환

개발자 통제 영역

사용자의 목이 통제하는 영역 — 직접 조작 금지



이동은 XR Origin 으로. Main Camera 의 Transform 을 직접 조작하면

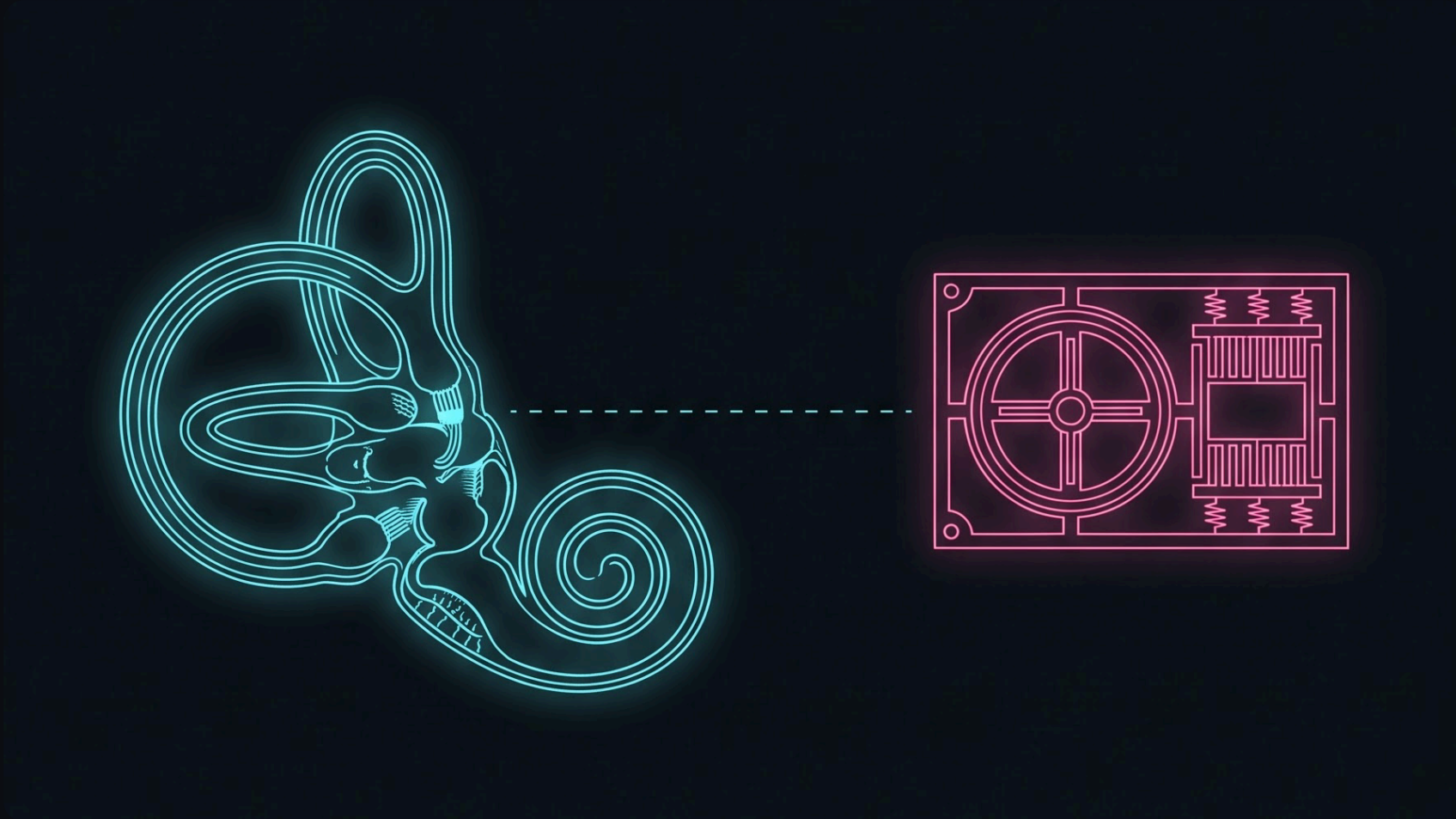
트래킹 값과 충돌 → 화면 떨림, 사용자 멀미 유발 ← 11주차 AI 함정 #8

원점 위치 선택

트래킹 모드	원점	용도
Floor (Stage)	바닥 중앙	서서 움직이는 콘텐츠 · 키 자동 반영
Device (Eye Level)	부팅 시 머리 위치	앉아서 하는 콘텐츠 · 리센터 필요

모드 선택 오류 → 키 180cm인 사람이 바닥에 파묻히거나 공중에 뜬
코드보다 설정 한 줄이 만드는 사고

전정계 — 몸에 달린 IMU



왼쪽 · 사람의 내이

서로 직각인 반고리관 3개 = 회전 감지 · 이석기관 = 중력과 직선 가속 감지

오른쪽 · MEMS IMU 칩

자이로스코프 + 가속도계 — 11주차 HMD 트래킹에 활용되는 부품

멀미의 원인

뇌 : 전정계가 감지한 움직임과 눈이 보는 움직임을 매
순간 대조

둘이 어긋나면 뇌는 독극물 중독으로 해석 → 구
토 반응

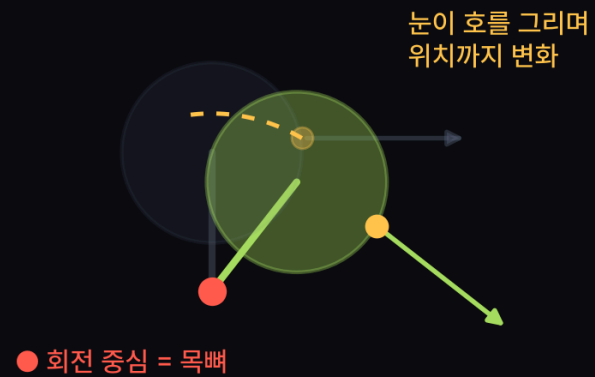
사이버 멀미의 원인 : 몸과 눈이 감지한 움직임의 불일치

	전정계	눈
실제로 걷기	움직임	움직임
영화 관람	정지	정지
VR 조이스틱 이동	정지	움직임

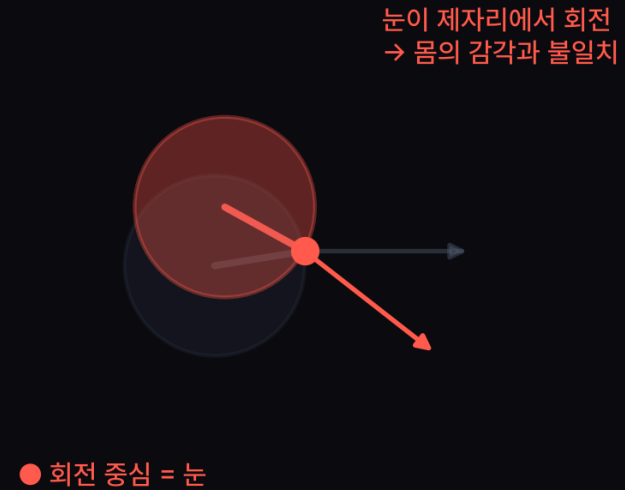
마지막 줄이 문제 → VR의 표준 이동 방식이 텔레포트가 된 이유 (이동 과정을 보여주지 않음)

회전 중심과 눈의 위치

실제 사람 — 목을 축으로 회전



순진한 구현 — 카메라를 제자리 회전



목뼈에서 눈까지 약 10 cm — 이 오프셋 누락 시 고개를 돌릴 때마다 세상이 미끄러짐

전정계가 그 어긋남을 정확히 감지 → 멀미 원인

고개 회전과 오늘 배운 변환

고개를 돌릴 때 눈의 움직임 = 회전 + 이동

눈의 새 위치 = $T(\text{목} \rightarrow \text{눈 오프셋}) \cdot R(\text{고개 회전}) \cdot \text{목 위치}$

← 오늘 배운 $R \cdot T$

오프셋(약 **10 cm**) 누락 시 고개를 돌릴 때마다 세상이 미끄러짐
전정계가 그 어긋남을 **정확히** 감지

Quest : 목 모델을 하드웨어에서 처리 — 카메라를 직접 건드리지 않는 한

설계 원칙

VR에서 움직이는 대상은 언제나 **리그(Rig)**
카메라 직접 이동 금지

멀미를 유발하는 코드

```
Camera.main.transform.position += v;
```

올바른 코드

```
xrOrigin.transform.position += v;
```

다음 주 예고

눈이 두 개 → 변환 체인의 끝이 두 갈래로 분기

Camera — $T(-IPD/2)$ → 왼쪽 눈 → 왼쪽 화면
 └ $T(+IPD/2)$ → 오른쪽 눈 → 오른쪽 화면

IPD (동공 간 거리) $\approx 58 \sim 72$ mm, 사람마다 다름

같은 씬을 두 번 렌더링 → 비용 두 배 → 프레임 예산 절반

PART III · 50분

프롬프트 실습

개념 이해 → 정확한 요구 → 오류 판별

실습의 3단 루프

1 개념

무엇을 만들어야 하는지 이론으로 이해

2 명세

정확한 용어로 시에게 요구
용어를 모르면 요구 불가

3 검증

결과가 이론적으로 맞는지 확인
틀렸다면 어디가 틀렸는지

실습의 핵심 : ②와 ③

코드 작성은 AI, 요구와 판별은 학생

AI 함정 #1 직접 재현

이렇게 요청하면

"큐브가 부드럽게 회전하게
스크립트를 만들어줘"

↓ AI의 응답

```
transform.eulerAngles +=  
    new Vector3(0, spd * dt, 0);
```

문제점

- `eulerAngles` 누적
- 매 프레임 쿼터니언 → 오일러 변환
- 값이 튀고 특정 각도에서 잠김
- 회전 중심 미지정

→ 학생이 지적해야 할 항목

명세 작성 예시

[목표]

큐브를 월드 Y축 기준으로 초당 45° 등속 회전시킨다.

[제약]

- 회전 누적에 `eulerAngles` 를 쓰지 말 것 (짐벌락·값 튼)
- Quaternion 을 유지한 채 누적할 것
- 프레임률과 무관하게 같은 속도 (`Time.deltaTime`)
- 회전 중심은 물체 자신의 원점

[검증 조건]

- 60fps 와 30fps 에서 10초 뒤 각도가 같을 것
- 어떤 각도에서도 축이 잠기지 않을 것

[비목표]

- 입력 처리, UI 는 만들지 말 것

네 칸 = 수업의 표준 명세 양식

[제약]과 **[검증 조건]**에 개념 이해도가 드러남

오늘의 실습 — 50분

#	실습	확인할 개념
1	위 명세로 회전 스크립트 생성, AI의 첫 응답을 그대로 기록	프롬프트 → 결과 대응
2	일부러 <code>eulerAngles</code> 버전을 만들어 짐벌락 재현, 영상 캡처	짐벌락 진단
3	부모-자식 3단 체인 구성, 부모에 <code>Scale(1,3,1)</code> 적용 후 자식의 찌그러짐 관찰	계층 변환 · 전단
4	Main Camera 직접 이동 코드와 부모 리그 이동 코드를 모두 작성해 비교	11주차 예비

1~3 필수(Core), 4 확장(Stretch) · 제출물 없음, 수업 안에서 완료

과제

이번 주 과제 — Meta 개발자 계정

Meta 개발자 계정 만들기

Quest 3 개발에 필요한 **개발자 인증** — 승인까지 시간 소요

11주차 실습 전 필수

제출 : 개발자 대시보드 캡처 · PLATO

미루면 11주차 실습 불가

결제 수단 등록·본인 확인 포함 → 승인까지 며칠 소요 가능, 가급적 오늘 안에 신청

한 장 요약

개념	핵심
내적	두 방향이 같은 쪽을 보는 정도 · 각도 비교는 코사인끼리
외적	두 벡터가 만드는 면의 법선 · 좌우 판정에도 활용
행렬	열 = 기저 벡터가 도착한 자리. 암기 대신 유도
동차좌표	한 차원 추가로 이동까지 흡수. $w=1$ 은 점, $w=0$ 은 방향
순서	$R \cdot T \neq T \cdot R$. 회전 중심 미지정 시 AI가 임의로 선택
짐벌락	오일러각의 구조적 결함 · 축이 겹치면 자유도 상실
쿼터니언	축 하나 + 각도 하나 · 순서가 없어 잠김 없음
VR 체인	카메라 대신 리그를 이동. 머리의 움직임은 사용자의 영역

다음 주

Unity 기초와 첫 바이트코딩

씬 그래프 · 컴포넌트 구조 · 게임 루프

AI 함정 #2 — 프레임롤에 따라 달라지는 속도